spinalSynth

Table of Contents

There is an additional document about the implementation details of this project for spinalHDL:

Implementation_specs.md

1. Introduction

This project implements a compact digital audio synthsizer in SpinalHDL.

The oscillator is based on Direct Digital Synthesis (DDS) using a phase accumulator architecture. The oscillator generates audio waveforms internally using an oversampled DDS engine and outputs stereo audio using the I²S protocol.

The project is intentionally designed to remain:

- compact
- deterministic
- FPGA-friendly
- easy to understand
- easy to simulate
- easy to extend later

Features

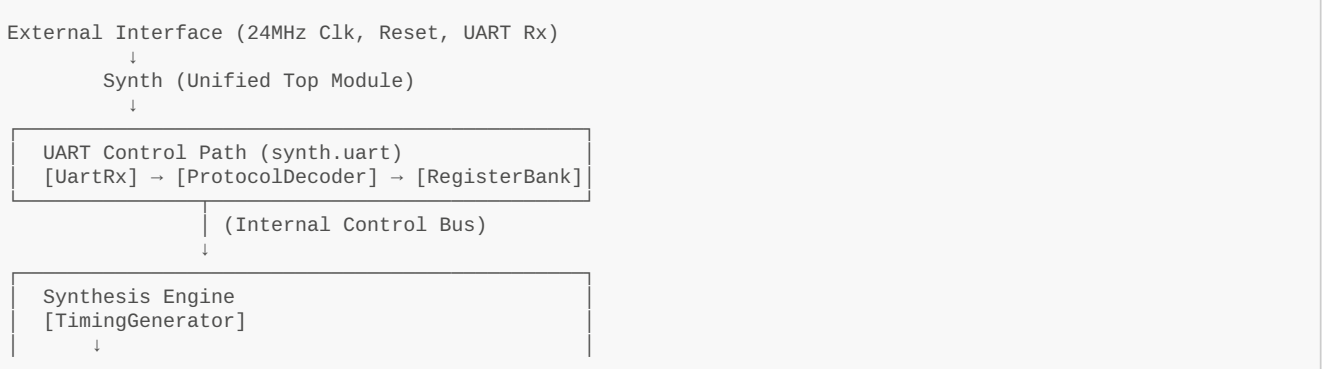
- 24-bit DDS phase accumulator
- 480 kHz internal DDS update rate
- 48 kHz stereo audio output
- 16-bit signed audio samples
- Stereo I²S output interface
- Oversampled waveform generation
- Single synchronous 24 MHz clock domain
- Clock-enable based timing architecture
- FPGA-friendly implementation

AI: ChatGPT and Gemini

The project was developed with the heavy usage of AI tools. All the specification documents were created via talking sessions to chatGPT, most of them in voice chat on the mobile with follow ups on the keyboard.

Impementation, debugging and testing happened in VSCode with the Gemini Extension.

2. High-Level Architecture



```
  │  [Oscillator] (synth.oscillator)              │
  │      ↓                                        │
  │  [Decimator]  (synth.output)                  │
  └──────────────────┬────────────────────────────┘
                     │ (48kHz Samples)
                     ↓
  ┌──────────────────────────────────────────────┐
  │  I2S Transmitter (synth.output)              │
  │  [BCLK] [LRCLK] [SDATA]                      │
  └──────────────────┬────────────────────────────┘
                     ↓
          Stereo Digital Audio
```

3. Master Clock

The complete design operates from a single synchronous master clock.

Parameter	Value
Master clock frequency	24 MHz

No internally-generated FPGA clocks shall be used.

All submodules shall operate synchronously from the 24 MHz master clock using clock-enable tick signals.

4. Timing Generators

The TimingGenerator module shall generate two independent clock-enable tick signals.

phaseTick

Parameter	Value
Frequency	480 kHz
Divider	24 MHz / 50
Purpose	Drive DDS phase accumulator

The phase accumulator and waveform generation logic shall update on this tick.

sampleTick

Parameter	Value
Frequency	48 kHz
Divider	24 MHz / 500
Purpose	Generate output audio samples

The decimator and output audio sample registers shall update on this tick.

5. Communication Protocol

The system is controlled via a standard UART interface. An external controller (such as a PC or Microcontroller) sends 3-byte packets to update the internal state of the synthesizer.

UART Configuration

Parameter	Value
Baud Rate	115,200
Data Bits	8
Parity	None
Stop Bits	1

Packet Format

The `UartProtocolDecoder` expects a 3-byte sequence for every command:

1. **Command Byte**: One byte for the command. (i.e. 0x01 for "write to register")
2. **Address Byte**: Specifies which register to write to.
3. **Data Byte**: The value to be written.

Command list

Right now there is only one command.

Command	Name	Adress Byte	Data Byte
`0x01`	`WriteRegister`	`From Register Map`	`1 Byte`

Register Map

Address	Register Name	Description	Width
`0x00`	`FREQ_LOW`	Frequency Word Bits [7:0]	8 bit
`0x01`	`FREQ_MID`	Frequency Word Bits [15:8]	8 bit
`0x02`	`FREQ_HIGH`	Frequency Word Bits [23:16] (Triggers Atomic Update)	8 bit
`0x03`	`WAVE_SEL`	0:Saw, 1:Square, 2:PWM, 3:Triangle, 4:Noise	3 bit
`0x04`	`PWM_WIDTH`	Duty cycle for PWM waveform	8 bit
`0x05`	`VOLUME`	Master output volume (Reserved)	8 bit

---

6. DDS Oscillator Architecture

The oscillator shall use a classic DDS architecture.

DDS Core

At every phaseTick:

```
phase := phase + freqWord
```

The phase accumulator shall wrap naturally on overflow.

Phase Accumulator

Parameter	Value
Width	24 bit
Type	Unsigned

Example:

```
val phase = Reg(UInt(24 bits))
```

Frequency Word

Parameter	Value
Width	24 bit
Type	Unsigned

The frequency word controls oscillator frequency.

Frequency Calculation

The DDS frequency equation is:

```
f = freqWord × updateRate / 2^24
```

Where:

Parameter	Value
updateRate	480 kHz
phase width	24 bit

Frequency Resolution

The minimum frequency step is:

```
480000 / 16777216 ≈ 0.0286 Hz
```

7. Waveform Generators

The oscillator shall support the following waveforms.

Saw

Generated by mapping the upper phase bits to audio amplitude.

Example:

```
sample = phase[23:8]
```

Square

Generated using the phase accumulator MSB.

Example:

```
if phase[23] == 1:
    +MAX
else:
    -MAX
```

PWM

Generated using a comparator between phase and pulseWidth.

The 8-bit PWM width value shall be expanded internally before comparison with the 24-bit phase accumulator.

The expansion shall be implemented by shifting the PWM value 16 bits to the left to match 24 bits width.

Example:

```
if phase < pulseWidth:
    +MAX
else:
    -MAX
```

PWM Width

Parameter	Value
Width	8 bit
Type	Unsigned

Triangle

Generated using reflected phase arithmetic.

To generate the triangle wave, we utilize a "reflected phase" technique based on the 24-bit phase accumulator. The Most Significant Bit (MSB) of the phase acts as a direction indicator: during the first half-cycle (MSB=0), the lower 23 bits create a linear rising ramp, whereas during the second half-cycle (MSB=1), those bits are bitwise inverted to produce a symmetrical falling ramp. This 23-bit result is then right-shifted by 7 bits to normalize it to a 16-bit range and cast to a signed integer (SInt), resulting in a full-swing bipolar waveform that transitions smoothly between peak amplitudes.

Noise

Noise generation shall use an LFSR-based pseudo-random generator.

Parameter	Value
Generator type	Fibonacci LFSR
Width	23 bit
Polynomial	x^23 + x^18 + 1
Feedback taps	bit 22 XOR bit 17
Update timing	`phaseTick`
Output type	16-bit signed
Reset seed	nonzero fixed value

An LFSR must never be initialized to zero, as it would stay stuck. We will plan to use a fixed non-zero seed.

The 16-bit signed audio is extracted just by taking the upper 16 bits of the LFSR.

---

8. Oversampling and Decimation

Oversampling Strategy

The DDS oscillator shall internally operate at:

```
480 kHz
```

while the final audio output sample rate shall be:

```
48 kHz
```

This creates an oversampling ratio of:

```
10×
```

Decimation Strategy

The implementation shall use simple zero-order decimation.

Every 10th DDS sample shall be captured as the output audio sample.

No interpolation or low-pass filtering shall initially be used.

Example:

```
if(sampleTick) {
    audioSample := oscSample
}
```

9. Audio Sample Format

Parameter	Value
Audio width	16 bit
Sample format	Signed
Sample rate	48 kHz

Example:

```
val sample = SInt(16 bits)
```

The oscillator is currently mono internally.

The mono signal shall be duplicated to both stereo output channels.

Example:

```
leftSample  = sample
rightSample = sample
```

10. I²S Output Interface

The output interface shall use the I²S protocol.

I²S Timing Architecture

The I²S transmitter shall operate directly from the 24 MHz master clock. The transmitter shall use a cycle-timed state machine architecture.

I²S Bit Timing

The required I²S bit clock frequency BCLK is:

```
48,000 × 2 × 16 = 1.536 MHz
```

The relationship to the 24 MHz master clock is:

```
24 MHz / 1.536 MHz = 15.625
```

Therefore no integer divider exists.

The serializer shall therefore alternate between:

- 15 master-clock cycles
- 16 master-clock cycles

between serialized bit transfers.

I²S Timing Subpattern

The serializer shall use the following repeating 8-step timing subpattern:

```
16,16,15,16,16,15,16,15
```

This subpattern contains:

Interval	Count
16-cycle intervals	5
15-cycle intervals	3

Total clocks:

```
16+16+15+16+16+15+16+15 = 125
```

Average clocks per bit:

```
125 / 8 = 15.625
```

This exactly matches the required average I²S bit timing.

Relationship To LRCLK Period

One stereo I²S frame contains:

```
32 serial bits
```

because:

- 16 left-channel bits
- 16 right-channel bits

Since:

```
32 = 4 × 8
```

the 8-step timing subpattern repeats exactly four times during one complete stereo frame.

Full frame timing:

```
[16,16,15,16,16,15,16,15] × 4
```

Total master-clock cycles per stereo frame:

```
4 × 125 = 500
```

Stereo frame rate:

```
24 MHz / 500 = 48 kHz
```

This produces the exact required audio sample rate.

I²S Timing State Machine

The serializer shall internally contain:

Register	Purpose
cycleCounter	Current interval countdown
patternIndex	Selects 15/16-cycle interval
bitCounter	Counts serialized bits
shiftRegister	Serialized audio data

The pattern index shall cycle continuously:

```
0 → 1 → 2 → ... → 7 → 0
```

The bit counter shall cycle:

```
0 → 1 → 2 → ... → 31 → 0
```

The bit counter determines:

- LRCLK state
- stereo frame boundaries
- sample reload timing

I²S Audio Format

Parameter	Value
Channels	2
Audio width	16 bit
Sample rate	48 kHz
Bit clock	1.536 MHz

I²S Signals

Signal	Description
i2s_bclk	Bit clock
i2s_lrclk	Left/right word select
i2s_sdata	Serial audio data

Serializer Behavior

The I²S serializer shall:

- shift audio data
- serialize stereo audio samples
- generate LRCLK framing
- output signed 16-bit audio samples

The exact serializer state machine behavior is not yet specified.

11. Numeric Formats

Signal	Type
phase	UInt(24 bits)
freqWord	UInt(24 bits)
pulseWidth	UInt(8 bits)
audioSample	SInt(16 bits)

The design shall use fixed-point arithmetic throughout.

12. Module Hierarchy

```
Synth
 ├── uart/ (Control Path)
 │     ├── UartRx
 │     ├── UartProtocolDecoder
 │     └── RegisterBank
 │
 ├── TimingGenerator
 │
 ├── oscillator/ (Core Engine)
 │     └── Oscillator
 │           ├── Accumulator
 │           ├── Generators
 │           ├── Noise
 │           └── Mux
 │
 └── output/ (Output Pipeline)
       ├── Decimator
       └── I2STransmitter
```

13. Confirmed System Parameters

Parameter	Value
HDL	SpinalHDL
Master clock	24 MHz
DDS phase width	24 bit
DDS update rate	480 kHz
Audio sample rate	48 kHz
Audio width	16 bit signed
I²S output	Stereo
I²S bit clock	1.536 MHz
Oversampling ratio	10×
Decimation method	Every 10th sample
Arithmetic	Fixed-point
Waveforms	Saw, Square, PWM, Triangle, Noise
Clocking strategy	Single synchronous clock domain